



Developed By : Abdul Wasay

DEPARTMENT OF COMPUTER SCIENCES

COMSATS UNIVERSITY ISLAMABAD, WAH CAMPUS

WAH CANTT – PAKISTAN

Institute Training : Dev Kaiens Wah Cantt





CyberShield

Security Analysis Platform

Project Report | Version 1.0.0 | May 2026

Full-Stack Cybersecurity Suite

Node.js (Express) Backend · Vanilla HTML/CSS/JS Frontend · 7 Security Modules

7	18	9	~2,000
Security Modules	API Endpoints	Frontend Pages	Lines of Code

1. Executive Summary

CyberShield is a full-stack, professional-grade cybersecurity analysis suite built with **Node.js (Express)** on the backend and **vanilla HTML/CSS/JavaScript** on the frontend. The platform consolidates **seven distinct security tools** into a single unified interface, ranging from real-time password breach detection to a live Intrusion Detection System (IDS). It is designed to demonstrate real-world cybersecurity concepts and is suitable as a portfolio or professional demonstration project.



Figure 1: CyberShield Dashboard — Cybersecurity Analysis Platform Hero

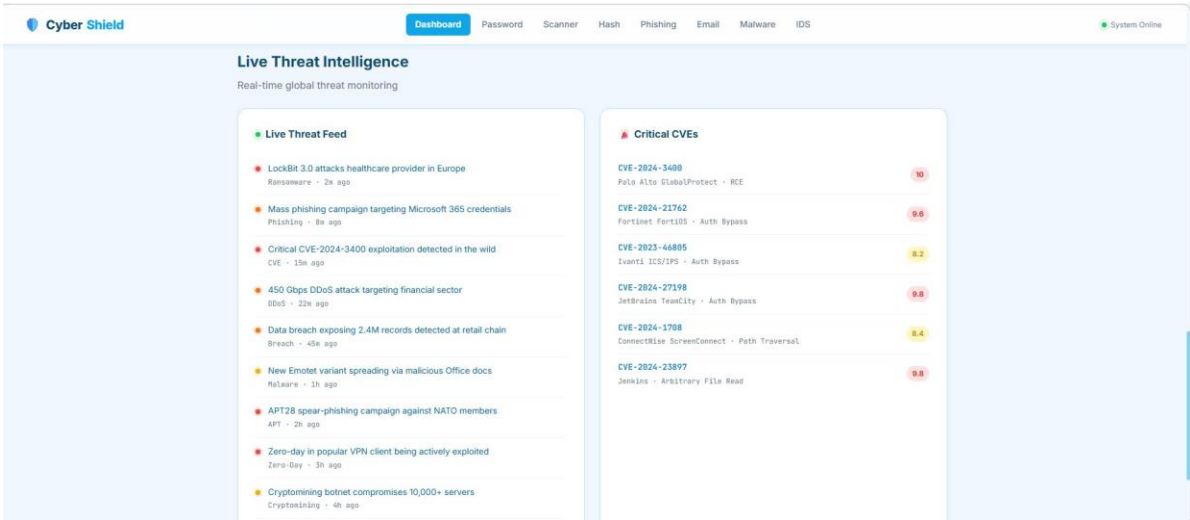


Figure 2: Live Threat Intelligence Feed with Critical CVEs

Platform Highlights

- Real-time password breach detection via HaveIBeenPwned k-anonymity API
- Active TCP socket network port scanning with CVE mapping
- Heuristic phishing URL analysis engine

- Cryptographic hash generation (MD5, SHA-1, SHA-256, SHA-512, Bcrypt)
- Malware file hash scanner with signature database
- Email header SPF/DKIM/DMARC authentication analyzer
- Live Intrusion Detection System with honeypot login and brute-force blocking

2. Project Structure

The project follows a clean separation of concerns with backend routes and frontend pages organized by feature module.

Component	Details
backend/server.js	Express app entry point, middleware, health check
backend/routes/auth.js	IDS / Honeypot login engine
backend/routes/email.js	Email header analysis engine
backend/routes/hash.js	Cryptographic hash & HMAC engine
backend/routes/malware.js	File hash scanner (signature DB)
backend/routes/password.js	Password strength + HIBP breach check
backend/routes/phishing.js	URL phishing heuristic engine
backend/routes/scanner.js	Real TCP network port scanner
backend/routes/threat.js	Live threat intelligence feed
frontend/index.html	Main dashboard
frontend/password.html	Password Analyzer UI
frontend/scanner.html	Network Scanner UI
frontend/hash.html	Hash Generator UI
frontend/phishing.html	Phishing Detector UI
frontend/malware.html	Malware Scanner UI
frontend/email.html	Email Header Analyzer UI
frontend/login.html	IDS Honeypot Login UI
frontend/admin.html	IDS Admin Dashboard UI
frontend/css/main.css	Shared design system

Total Files: 19 source files | Backend Size: ~51 KB | Frontend Size: ~58 KB

3. Technology Stack

Layer	Technology	Purpose
Runtime	Node.js v18+	JavaScript server runtime
Framework	Express.js v4.18	HTTP server & routing
Security	Helmet.js v8.1	Secure HTTP headers
Rate Limiting	express-rate-limit v7.1	API abuse protection
Cryptography	Node.js crypto module	Hashing, HMAC, timing-safe compare
Password Hashing	bcrypt v6.0	Secure password hash generation
File Upload	Multer v2.1	In-memory multipart file handling
HTTP Client	Native fetch (Node 18+)	HIBP API integration
Network	Node.js net + dns	Real TCP socket scanning & DNS lookup
Frontend	HTML5 / Vanilla CSS / JS	Zero-dependency UI
Fonts	Google Fonts (Inter, JetBrains Mono)	Professional typography
CORS	cors v2.8	Cross-origin request handling

4. Backend Architecture

4.1 Server Configuration (server.js)

The Express server is configured with enterprise-grade middleware:

- **Helmet.js** — Sets secure HTTP response headers (XSS protection, HSTS, etc.)
- **CORS** — Allows cross-origin requests for local development
- **Rate Limiting** — 200 requests / 15 minutes per IP on all /api/* routes
- **Static File Serving** — Serves the frontend/ directory directly
- **Global Error Handler** — Catches and logs all unhandled errors

Server Port: 3000 (configurable via process.env.PORT)

Health Check Endpoint: GET /api/health

```
{ "status": "online", "version": "1.0.0", "uptime": "312.45s" }
```

4.2 Route Modules

Route Prefix	File	HTTP Methods	Key Endpoints
/api/password	password.js	POST	/analyze, /generate
/api/hash	hash.js	POST, GET	/generate, /compare, /hmac, /encode, /algorithms
/api/scanner	scanner.js	POST, GET	/scan, /ports
/api/threat	threat.js	GET	/dashboard, /feed, /actors
/api/phishing	phishing.js	POST	/analyze
/api/email	email.js	POST	/analyze
/api/malware	malware.js	POST	/scan
/api/auth	auth.js	POST, GET	/login, /logs

5. Module Deep-Dives

5.1 Password Analyzer (/api/password)

Purpose: Analyzes password strength using multiple real metrics and checks for real-world data breaches.

Core Algorithms:

- Shannon Entropy — Measures character randomness ($H = -\sum p(x) \log_2 p(x)$)
- Charset Pool Size — Counts contributing character sets (lowercase: 26, uppercase: 26, digits: 10, symbols: 32)
- Brute-Force Time Estimation — Based on combinations = $\text{charsetSize}^{\text{length}}$ at 10 billion guesses/sec (modern GPU speed)
- Pattern Detection — Detects repeated chars, keyboard patterns (qwerty, asdf), sequential numbers, and common admin terms
- Strength Score — 0-100 composite score across 9 weighted checks

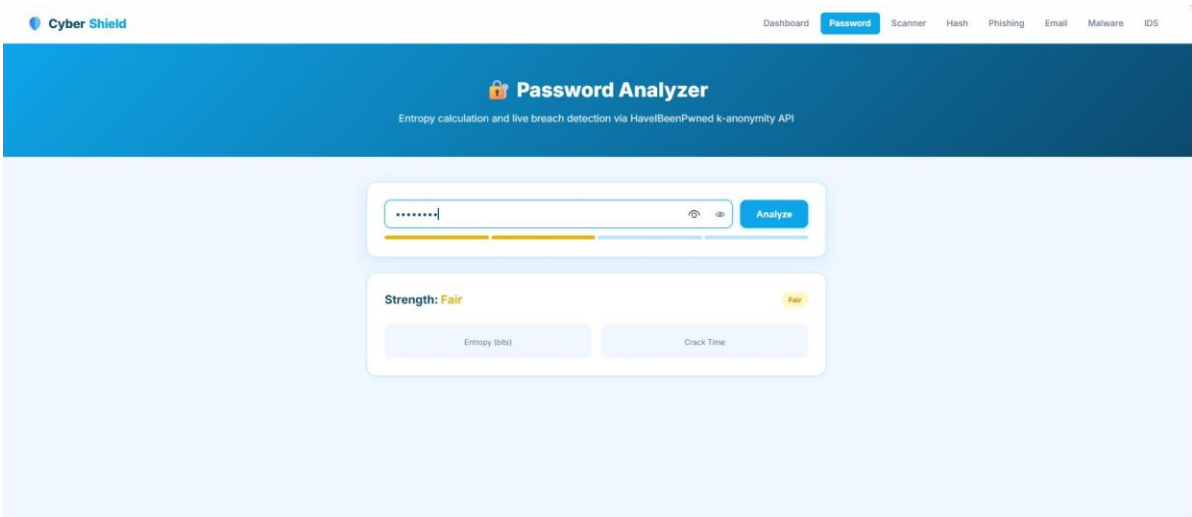


Figure 3: Password Analyzer — Fair Strength (medium password)



Figure 4: Password Analyzer — Strong Strength (complex password)

HIBP Integration (Real Breach Check):

Uses the k-Anonymity model with the HaveIBeenPwned API:

- SHA-1 hash the password
- Send only the first 5 hex characters to `api.pwnedpasswords.com/range/{prefix}`
- Match the returned suffix list locally — password is NEVER sent over the network

Sample API Response:

```
{ "score": 85, "strength": { "label": "Very Strong", "color": "#00ff88" },  
  "entropy": 4.521, "crackTime": { "time": "14+ centuries", "severity": "safe" },  
  "breach": { "found": false, "count": 0 } }
```

5.2 Network Port Scanner (/api/scanner)

Purpose: Performs real TCP socket connections to scan 20 common ports on any IP or domain.

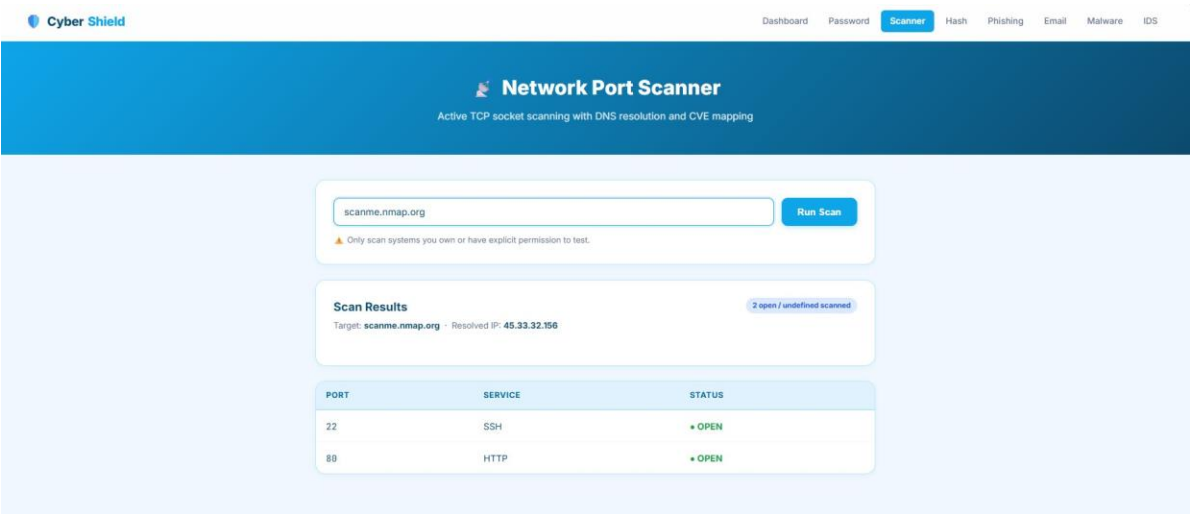


Figure 5: Network Scanner — Scanning scanme.nmap.org (Ports 22, 80 Open)

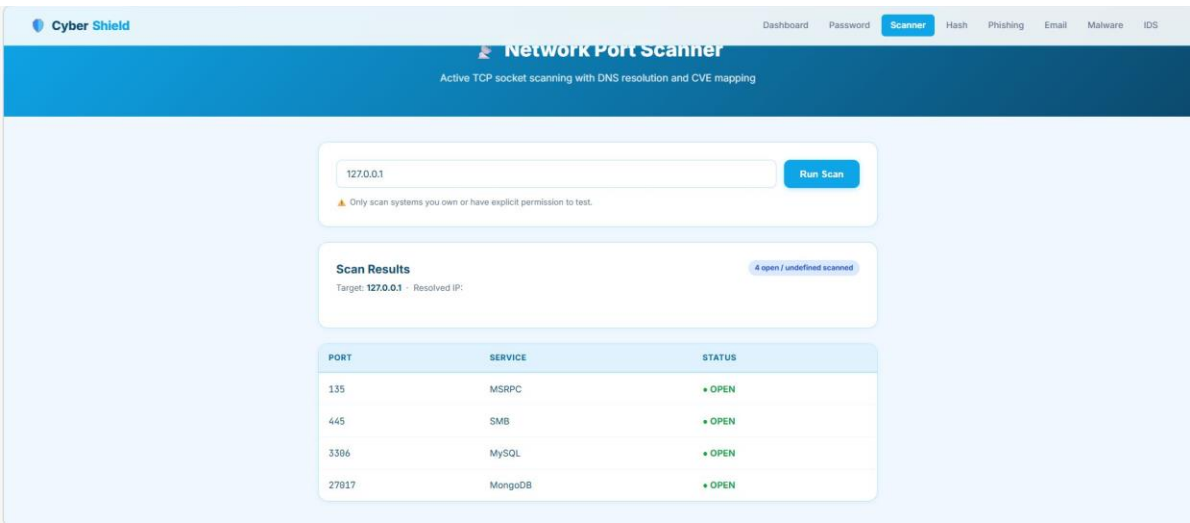


Figure 6: Network Scanner — Localhost Scan (Multiple Ports Open: MSRPC, SMB, MySQL, MongoDB)

How It Works:

- Uses Node.js net.Socket for real TCP connect scanning (not simulated)
- Each port gets a 2-second timeout — timeout implies firewall filtering
- All 20 ports scanned in parallel using Promise.all()
- DNS resolution via Node.js dns.promises for domain targets

Ports Scanned:

Port	Service	Notable CVEs
21	FTP	Brute-force, anonymous access
22	SSH	CVE-2023-38408 (OpenSSH)
80 / 443	HTTP / HTTPS	General web vulnerabilities
135	MSRPC	CVE-2003-0352 (MS RPC DCOM)
445	SMB	CVE-2017-0144 (EternalBlue, CVSS 9.3)
3306	MySQL	CVE-2012-2122, auth bypass
3389	RDP	CVE-2019-0708 (BlueKeep, CVSS 9.8)
6379	Redis	CVE-2022-0543 (Lua RCE, CVSS 10.0)
27017	MongoDB	Unauthenticated access

Risk Scoring Formula:

```
Risk Score =  $\Sigma(\text{port\_weight}) + (\text{critical\_CVE\_count} \times 15)$   
Normalized to 0-100 | Low (<20) | Medium (<40) | High (<70) | Critical (≥70)
```

5.3 Hash Generator & Verifier (/api/hash)

Purpose: Multi-algorithm cryptographic hashing, hash comparison, HMAC generation, and Base64 encoding.

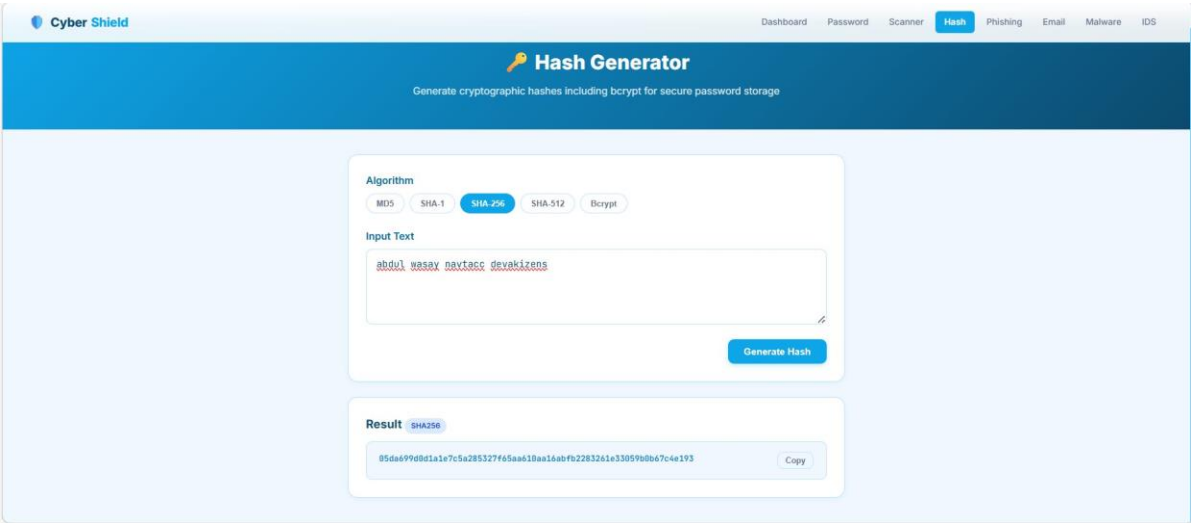


Figure 7: Hash Generator — SHA-256 Hash Generation

Algorithm	Bits	Security Status
MD5	128	Broken — educational only
SHA-1	160	Deprecated — collision vulnerable
SHA-256	256	Widely used, secure
SHA-384	384	Truncated SHA-512, secure
SHA-512	512	High security, large output
SHA3-256	256	Next-gen hashing standard
Bcrypt	192	Password hashing (slow by design)

Key Features:

- Timing-Safe Comparison — Uses `crypto.timingSafeEqual()` to prevent timing attacks
- HMAC Generation — Keyed-hash message authentication code for data integrity
- Base64 Encode/Decode — Utility encoding operations
- Bcrypt — Async generation with configurable salt rounds (default: 10)

5.4 Phishing URL Detector (/api/phishing)

Purpose: Heuristic-based URL analysis to detect phishing indicators without external API calls.

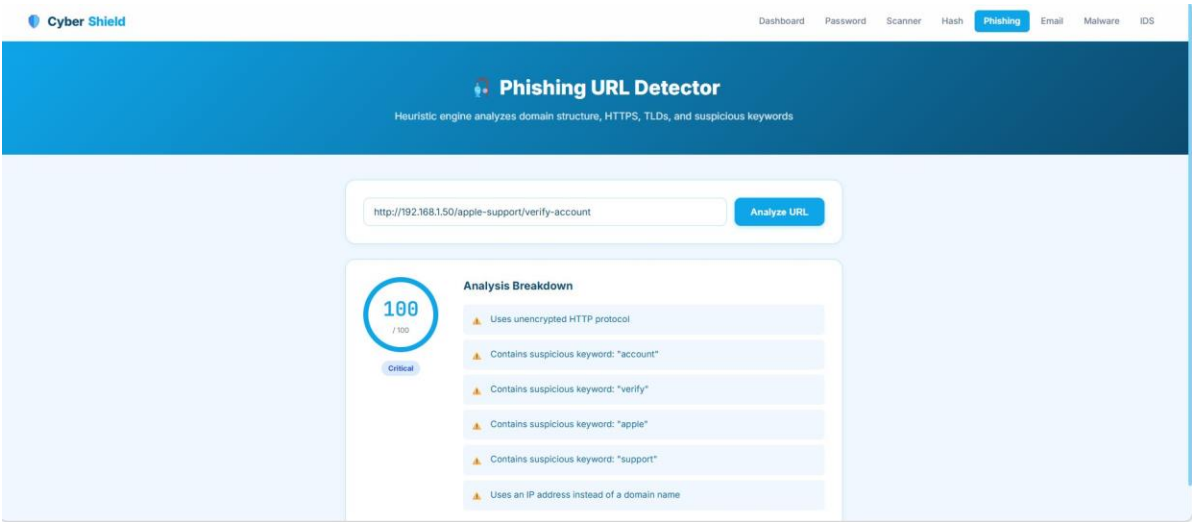


Figure 8: Phishing Detector — Critical Score (100/100) for IP-based phishing URL

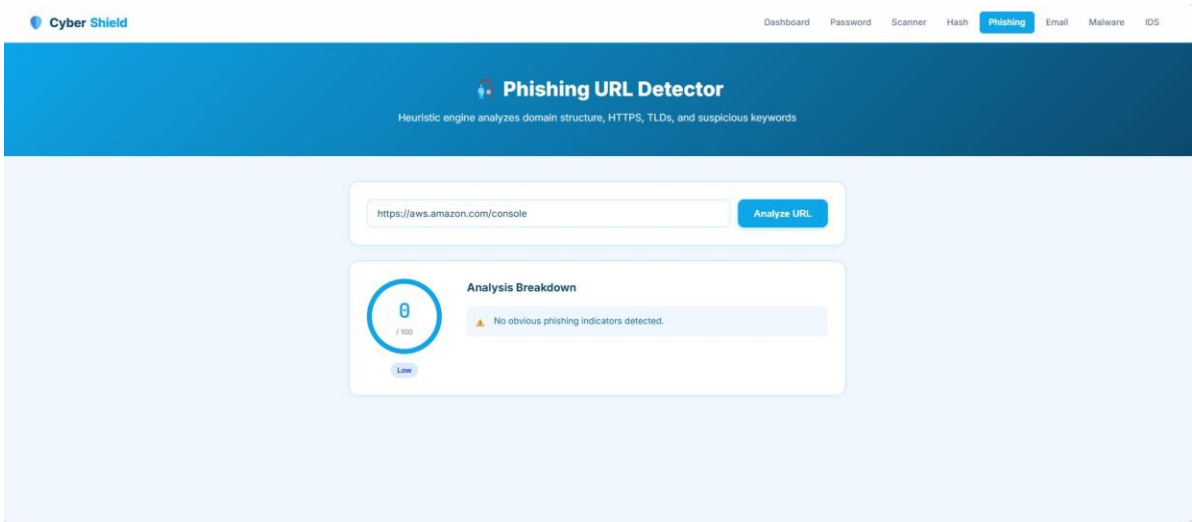


Figure 9: Phishing Detector — Low Score (0/100) for legitimate AWS URL

Heuristic Check	Score Added	Reason
No HTTPS	+20	Unencrypted protocol
Suspicious keywords in URL	+15 each	login, secure, paypal, microsoft, etc.
High-risk TLD	+25	.xyz, .top, .tk, .ml, etc.
IP address as host	+30	Bypasses domain reputation checks
Excessive subdomains (>4)	+15	Common obfuscation tactic

Heuristic Check	Score Added	Reason
URL length > 75 chars	+10	Hides real domain in long string
Hyphen in domain	+5	Common in phishing (paypa-l.com)
@ symbol in URL	+30	Used to spoof destination

Threat Levels: Low (0-29) → Medium (30-59) → High (60-79) → Critical (80-100)

5.5 Malware File Scanner (/api/malware)

Purpose: Uploads a file, computes its SHA-256 hash, and matches against a known malware signature database.

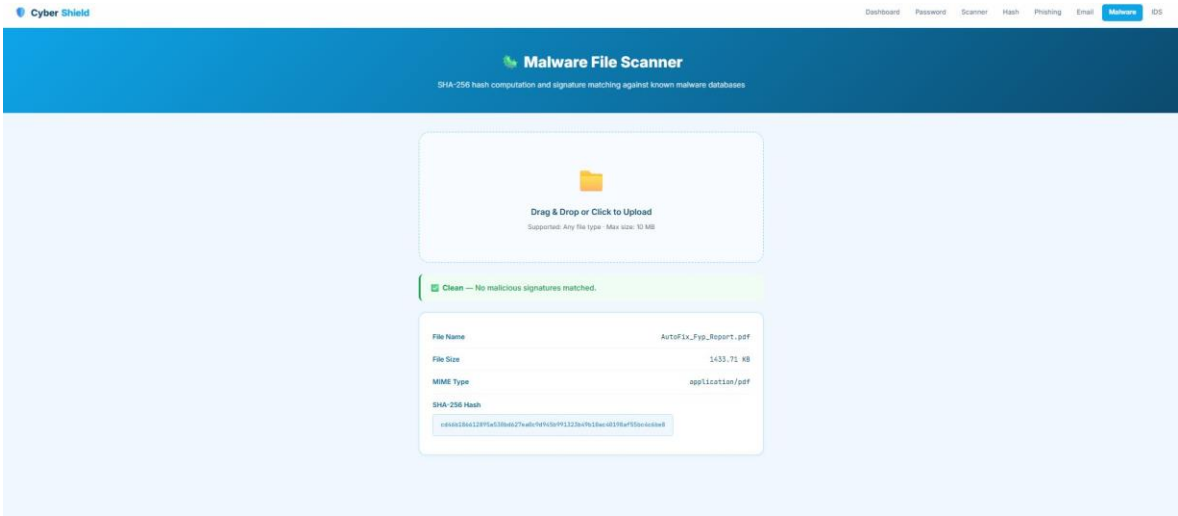


Figure 10: Malware File Scanner — Clean File Result with SHA-256 Hash

How It Works:

- File uploaded in-memory via Multer (max 10MB, no disk write)
- SHA-256 hash computed using Node.js crypto module
- Hash compared against hardcoded + expandable signature database
- Extension analysis for suspicious file types (.exe, .bat, .ps1, .vbs, .dll, etc.)

SHA-256 Hash (prefix)	Malware Name	Type
275a021b...	EICAR-Test-File	Test Virus
24d004a1...	Ransom:Win32/WannaCrypt	Ransomware
e3b0c442...	Trojan:Win32/Emotet	Banking Trojan

5.6 Email Header Analyzer (/api/email)

Purpose: Parses raw email headers to detect spoofing via SPF, DKIM, and DMARC analysis.

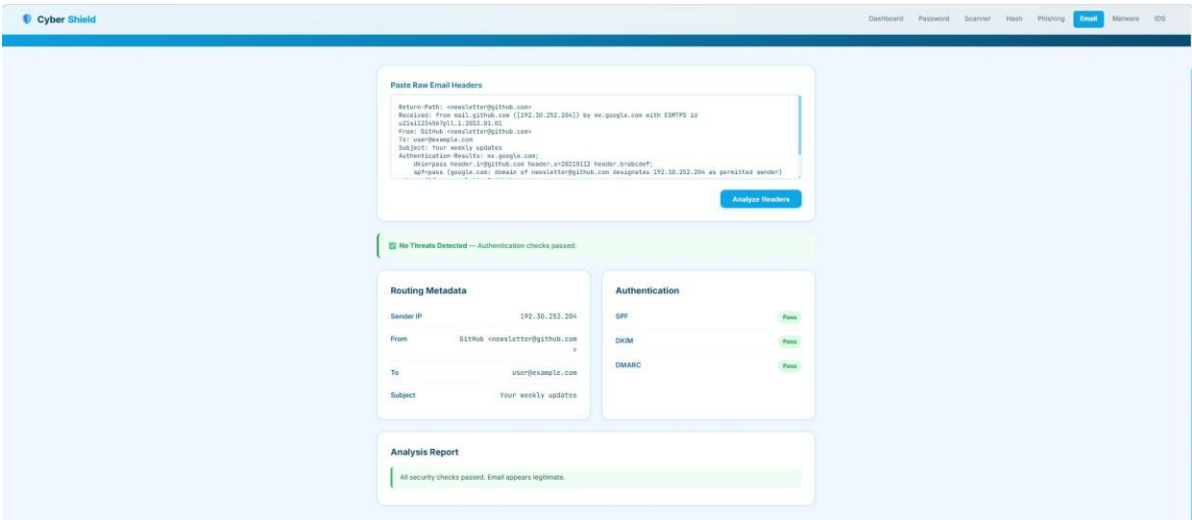


Figure 11: Email Header Analyzer — Clean GitHub Email (SPF/DKIM/DMARC: Pass)

Parsed Fields:

- From, To, Subject — Email metadata
- Received — Extracts originating IP address via regex
- Authentication-Results — Extracts SPF/DKIM/DMARC results

Failure Type	Score Added
SPF Fail	+30
SPF Unknown	+10
DKIM Fail	+30
DKIM Unknown	+10
DMARC Fail	+30
DMARC Unknown	+10
Display name spoofing	+40

Folded Header Support: Properly re-joins RFC 2822 folded header lines before parsing.

5.7 Login IDS — Intrusion Detection System (/api/auth)

Purpose: A honeypot login endpoint that simulates brute-force detection with IP-based blocking and a live attack log dashboard.

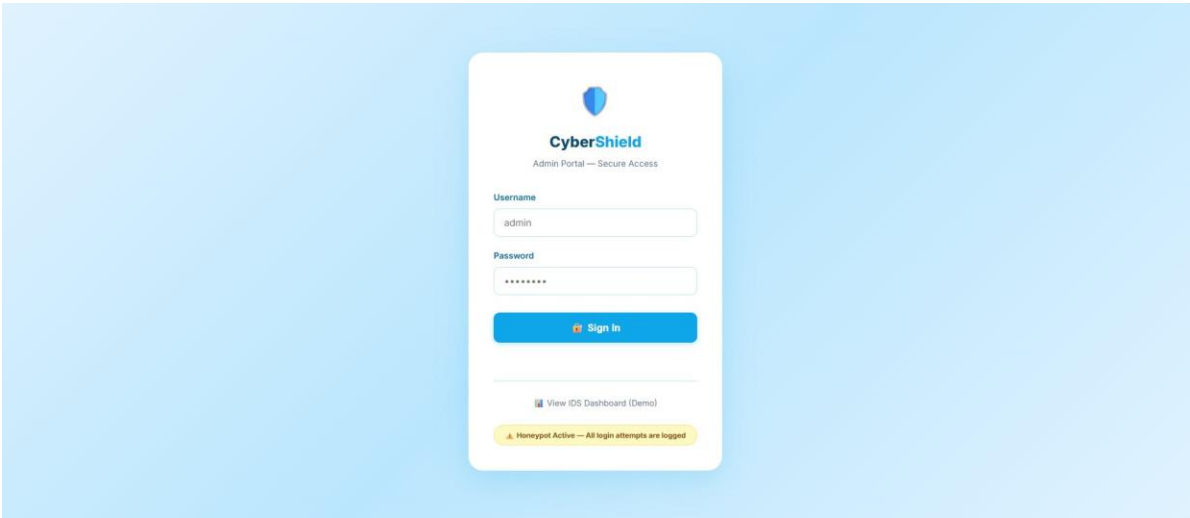


Figure 12: Honeypot Login Page — IDS Active Warning

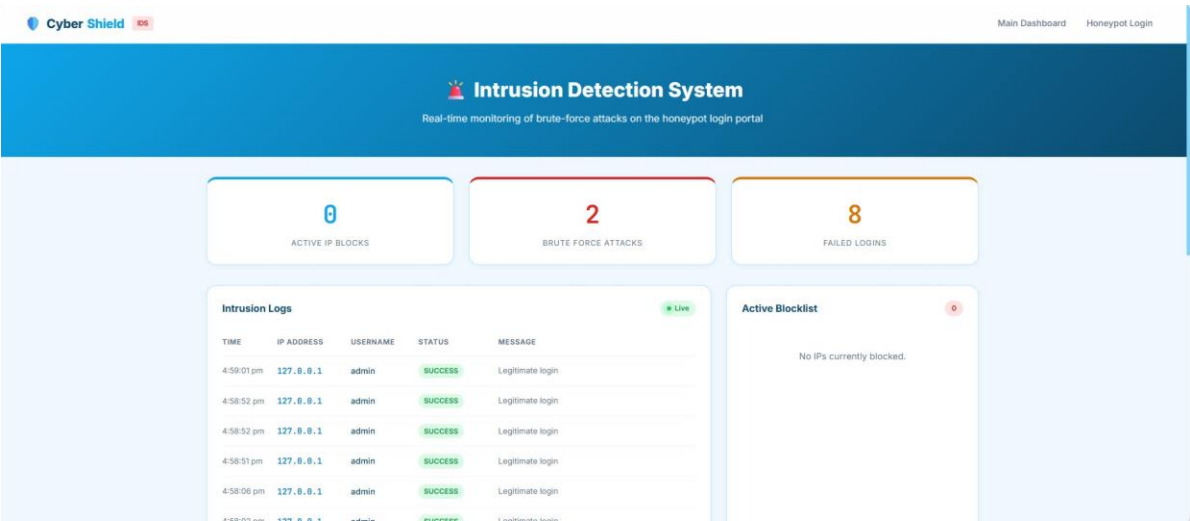


Figure 13: IDS Admin Dashboard — Live Intrusion Logs & Block List

IDS Logic:

- All login attempts are logged with IP, username, timestamp, and status
- After 5 failed attempts, the offending IP is blocked for 60 seconds
- Block list auto-expires and clears on next request from that IP
- Only one valid credential set exists (admin / CyberShield2026!) — all others trigger logging

Status Types Logged: SUCCESS | FAILED | BRUTE_FORCE | BLOCKED

Sample Admin Dashboard API Response:

```
{ "logs": [...],  
  "blockedIPs": [{ "ip": "127.0.0.1", "expiresIn": "45s" }],  
  "stats": { "totalAttacks": 3, "currentlyBlocked": 1, "recentFailures": 7 } }
```

6. Security Features

Feature	Implementation
Secure HTTP Headers	Helmet.js (X-Frame-Options, HSTS, XSS-Protection, etc.)
Rate Limiting	200 req / 15 min per IP on all /api/* routes
Timing-Safe Hash Compare	crypto.timingSafeEqual() prevents timing attacks
k-Anonymity Breach Check	Password SHA-1 prefix sent — full hash never leaves server
IP-Based Brute Force Block	In-memory Map; auto-expires after 60 seconds
File Upload Size Limit	10MB max; stored in memory only (no disk write)
Input Validation	All endpoints validate and sanitize input before processing
JSON Body Limit	10mb max request body via Express

7. API Reference Summary

Method	Endpoint	Description
GET	/api/health	Server health and uptime
POST	/api/password/analyze	Analyze password strength + breach check
POST	/api/password/generate	Generate a secure random password
POST	/api/hash/generate	Generate hashes for input text
POST	/api/hash/compare	Compare hash with timing-safe equality
POST	/api/hash/hmac	Generate HMAC with secret key
POST	/api/hash/encode	Base64 encode/decode
GET	/api/hash/algorithms	List all supported algorithms
POST	/api/scanner/scan	Real TCP port scan on target
GET	/api/scanner/ports	Port CVE database
GET	/api/threat/dashboard	Full threat intelligence data
GET	/api/threat/feed	Live threat event feed
GET	/api/threat/actors	Known threat actor profiles
POST	/api/phishing/analyze	Analyze URL for phishing indicators
POST	/api/email/analyze	Parse email headers for spoofing
POST	/api/malware/scan	Scan uploaded file hash vs signature DB
POST	/api/auth/login	IDS honeypot login endpoint
GET	/api/auth/logs	IDS attack log and blocked IPs

8. Running the Project

Prerequisites

- Node.js v18+ (required for native fetch)
- npm v8+

Installation & Start

```
cd CyberShield/backend
npm install
npm start          # Production
npm run dev        # Development (nodemon auto-reload)
```

Access URLs

Page	URL
Dashboard	http://localhost:3000
Password Analyzer	http://localhost:3000/password
Network Scanner	http://localhost:3000/scanner
Hash Generator	http://localhost:3000/hash
Phishing Detector	http://localhost:3000/phishing
Email Analyzer	http://localhost:3000/email
Malware Scanner	http://localhost:3000/malware
IDS Honeypot Login	http://localhost:3000/login
IDS Admin Dashboard	http://localhost:3000/admin

9. Codebase Metrics

Metric	Value
Total Source Files	19
Backend Routes	8 modules
API Endpoints	18
Frontend Pages	9 HTML pages
Backend LOC (approx.)	~1,050 lines
Frontend LOC (approx.)	~950 lines
npm Dependencies	6 production, 1 dev
Ports Scanned Per Scan	20
CVEs in Scanner DB	18 real CVEs
Threat Actors in DB	8

10. Future Roadmap

Feature	Priority	Notes
VirusTotal API Integration	High	Real-time malware hash lookup
JWT Authentication	High	Protect the admin dashboard
Persistent Logging (SQLite/MongoDB)	Medium	IDS logs survive server restarts
WHOIS / GeoIP Lookup	Medium	Add to scanner/phishing results
Real DKIM DNS Lookup	Medium	Verify signatures against live DNS
Shodan API Integration	Low	Enrich port scan with internet-wide data
Docker Support	Low	One-command containerized deployment
Export Reports (PDF/JSON)	Low	Download scan results
User Accounts & History	Low	Save past scans per user
WebSocket Live Updates	Low	Real-time IDS alert push to admin

11. Disclaimer

⚠ For Authorized Educational and Professional Use Only The network scanner performs real TCP connections. Only scan systems you own or have explicit written permission to test. Unauthorized port scanning may violate laws in your jurisdiction.

Report generated: May 10, 2026 | CyberShield v1.0.0